

Clustering et Détection d'Anomalies

Groupe constitué de :

- Pathy Jane Noutcha Ngounou
- Luc Nathan Ramasamy
- Abdoul Malick Oumarou Allassane
- Owen Couturier

Préparation des données	1
Clustering.....	4
DBSCAN	5
K-Means	6
Spectral.....	6
Détection d'anomalies.....	7
Auto-Encodeurs (AE)	8
Isolation Forest (IF).....	10
Mécanisme d'Isolation et Architecture	10
Paramètres et Implémentation	11
Processus d'Entraînement	11
Avantages Computationnels	11
Résultats Expérimentaux	11
Analyse de Stabilité et Compromis	12
Avantages Distinctifs	12
Limitations et Considérations	12
Recommandations d'Application.....	12
Architecture Hybride et Perspectives	13
Conclusion	13

Préparation des données

La phase de préparation des données constitue une étape essentielle du pipeline analytique, car elle garantit la qualité, la cohérence et la représentativité des informations avant l'application des algorithmes de clustering et de détection d'anomalies. Dans ce projet, deux jeux de données sont utilisés : **Hi-Seq** (expression génique) et **ECG** (activité cardiaque). Les opérations de préparation ont été centralisées dans le fichier **preprocess.py**, et les visualisations de base dans **utils.py**.

La préparation se déroule en plusieurs étapes :

1. Chargement des jeux de données :

Le dataset Hi-Seq est constitué de 20 531 variables génétiques et de cinq classes de tumeurs (BRCA, KIRC, COAD, LUAD, PRAD). Le chargement inclut : lecture du fichier `hiseq_data.csv`, la définition de l'ID d'échantillon comme index, la lecture des labels (Class), la fusion des deux sources en un seul tableau. L'ensemble est réalisé via la fonction `load_hiseq()`.

Le fichier `ecg.npz` contient 4 998 signaux ECG, chacun représenté par 141 attributs. Le chargement extrait : les 140 premières colonnes, la dernière colonne → **label (0 : normal, 1 : anomalie)**. Cela est fait via la fonction `load_ecg()`.

2. Nettoyage :

Le nettoyage vise à éliminer les incohérences et les variables inutiles : **Étapes appliquées (fonction `clean()`)** Imputation des valeurs manquantes : remplacement par la médiane, Suppression des colonnes constantes (variables sans variance)

Cette étape améliore : la stabilité statistique, la performance des algorithmes de clustering, la robustesse des modèles de détection d'anomalies, Les deux jeux de données (Hi-Seq et ECG) sont systématiquement nettoyés.

3. Normalisation : Les variables sont ensuite normalisées via un StandardScaler : $X_{norm} = \frac{X - \mu}{\sigma}$

Ce choix est essentiel car :

- Hi-Seq contient des milliers de gènes avec des échelles très différentes
- Les méthodes de clustering (K-Means, Spectral) sont sensibles aux distances
- Les autoencodeurs nécessitent une distribution stable pour converger

La fonction utilisée est `normalize_data()`.

4. Réduction de dimension pour visualisation

Deux types de réductions sont appliquées :

4.1 PCA 2D (visualisation)

Le but est d'obtenir une vue globale des données dans un espace à deux dimensions pour inspecter visuellement les motifs ou séparations entre classes.

Les projections 2D sont utilisées pour :

- Vérifier s'il existe des structures intrinsèques
- Analyser la dispersion des classes
- Détecter des anomalies grossières

La fonction utilisée est `reduce_dimension_pca(..., n_components=2)`.

Les visualisations sont générées via `plot_pca_2d()`

4.2 PCA 100D (pour le clustering Hi-Seq)

Comme Hi-Seq contient plus de **20 000 dimensions**, la malédiction de la dimensionnalité empêche les algorithmes classiques d'être efficaces. Une PCA ramenée à **100 composantes** permet de :

- Réduire le bruit
- Concentrer l'information pertinente
- Accélérer fortement K-Means, DBSCAN et Spectral Clustering

Cette réduction est conforme à la directive du TP qui exige une segmentation avec 100 dimensions.

5. Création d'un pipeline complet Hi-Seq et ECG

5.1 Pipeline Hi-Seq

La fonction `prepare_hiseq()` effectue :

1. Chargement et fusion des données
2. Séparation X / y
3. Nettoyage
4. Normalisation
5. PCA 2D pour visualisation
6. PCA 100D pour clustering

La sortie contient :

- X_raw : données brutes nettoyées
- X_scaled : données normalisées
- X_pca2d : réduction à 2 dimensions
- X_pca100 : réduction à 100 dimensions
- les objets scaler, pca2d, pca100

5.2 Pipeline ECG

La fonction `prepare_ecg()` effectue :

1. Chargement des signaux
2. Nettoyage
3. Normalisation
4. PCA 2D pour visualisation (uniquement)

La réduction à 100 dimensions n'est pas nécessaire pour ECG car le dataset comporte seulement 140 attributs.

La phase de préparation des données a permis :

- de nettoyer et stabiliser les jeux de données
- d'harmoniser les échelles
- d'extraire des représentations plus compactes
- d'obtenir des visualisations utiles à l'interprétation
- de réduire la complexité pour les algorithmes de clustering

Cette étape fournit donc une base solide et cohérente pour les deux grandes parties suivantes:

Clustering et Détection d'anomalies, conformément aux attentes du TP2.

Clustering

Pour chaque méthode de clustering "classique", il faudra comparer :

1. une application sur l'ensemble des attributs,
2. une application sur 100 attributs projetés obtenus par ACP.

Pour chaque application, il faudra évaluer les performances sur les métriques statistiques citées ci-après et sur les métriques de performances machine en temps et en mémoire.

Métriques internes

- Coefficient de silhouette de -1 à $+1$: succès proche de 1
- Indice de Davies-Bouldin positif : succès proche de 0
- Indice de Calinski-Harabasz positif : succès pour une grande valeur

Métriques externes

- Indice de Rand ajusté -1 à $+1$: succès proche de 1
- Information mutuelle normalisée de 0 à 1 : succès proche de 1

De plus, chaque métrique sera évaluée en moyenne avec un écart-type sur 10 exécutions initialisées aléatoirement pour garantir leur indépendance statistique.

Dans le cas du dataset HiSeq, il y a 5 classes : BRCA (sein), KIRC (rein), COAD (côlon), LUAD (poumon) et PRAD (prostate)

DBSCAN

Pour rappel, l'approche DBSCAN utilise le principe de points centraux définis par deux paramètres "eps" et "min_samples" qui contrôlent respectivement le rayon de mesure et le nombre minimal de points dans ce rayon pour être considéré comme central. En particulier, cet algorithme permet de détecter des clusters dont la forme n'est pas circulaire, de mesurer le bruit, et ce sans spécifier le nombre de cluster au préalable.

Nous avons appliqué l'algorithme sur les données complètes normalisées dans un premier temps avec $\text{eps}=90$ et $\text{min_samples}=2$. Cet algorithme est déterministe donc l'écart-type est nul.

	silhouette	davies_bouldin	calinski_harabasz	adjusted_rand	nmi	time	space
Value	-0.1491 ± 0.0	1.9038 ± 0.0	3.0172 ± 0.0	0.0143 ± 0.0	0.0706 ± 0.0	0.7355 ± 0.0326	247.3609 ± 1.0807

Puis sur les données projetées par ACP:

	silhouette	davies_bouldin	calinski_harabasz	adjusted_rand	nmi	time	space
Value	0.0077 ± 0.0	2.3639 ± 0.0	26.5687 ± 0.0	0.2977 ± 0.0	0.5045 ± 0.0	0.1522 ± 0.0174	0.425 ± 1.2026

Globalement, on remarque une amélioration des métriques avec projection, tant en performances que sur les métriques statistiques hormis l'information mutuelle. Cependant, ces métriques semblent peu satisfaisantes et en examinant les clusters créés, on remarque une

influence importante des paramètres. De plus, dans notre l'analyse du bruit et la flexibilité sur le nombre de classe n'est pas nécessaire voire heurte la qualité des résultats. Une méthode comme K-Means pourrait être davantage adaptée pour ces raisons.

K-Means

K-Means cherche les K clusters de façon à minimiser l'inertie. On sait qu'on cherche à détecter 5 types de tumeurs donc on réitère l'expérience avec cet algorithme, sur les données complètes puis projetées.

	silhouette	davies_bouldin	calinski_harabasz	adjusted_rand	nmi	time	space
Value	0.1285 ± 0.0087	2.6234 ± 0.0672	61.71 ± 5.1759	0.7732 ± 0.0402	0.8336 ± 0.0307	0.4929 ± 0.0452	243.3652 ± 0.2198

Déjà, K-Means présente une amélioration sur toutes les métriques par rapport à DBSCAN, et sur les données projetées:

	silhouette	davies_bouldin	calinski_harabasz	adjusted_rand	nmi	time	space
Value	0.1993 ± 0.0172	2.0984 ± 0.0556	102.9134 ± 5.4193	0.7851 ± 0.0413	0.8484 ± 0.0247	0.229 ± 0.0833	0.1988 ± 0.5965

Là aussi, une amélioration sur toutes les métriques par rapport aux données complètes, avec un écart-type stable, excepté pour davies_bouldin. Ce qu'il y a de plus notable c'est le gain en mémoire. Alors, on peut conclure que la méthode K-Means est plus efficace et qui plus est, elle bénéficie de la projection ACP du fait de la malédiction des dimensions. Ceci tend à interroger sur des méthodes spécifiquement développées pour ce contexte de grandes dimensions, par exemple le clustering spectral qui intègre une projection spectrale particulière avant d'appliquer K-Means.

Spectral

On va construire une matrice de similarité par noyau gaussien puis calculer le Laplacien pour faire un clustering sur son spectre. Cette méthode de clustering spectral sert à obtenir des vecteurs propres (les plus petits) sur lesquels les données seront projetées de façon à minimiser la coupe du graphe pour un clustering k-means classique.

Dans notre cas, le clustering spectral de scikit-learn utilise son module de "SpectralEmbedding" pour la partie projection. Celui-ci renvoie un warning sur le nombre de composantes connexes:

```
```bash
```

```
sklearn/manifold/_spectral_embedding.py:328: UserWarning:
```

```
Graph is not fully connected, spectral embedding may not work as expected.
```

```
```
```

Il y a plusieurs composantes connexes, plusieurs 0 dans le spectre du Laplacien, ce qui peut poser des problèmes sur les clusters comme signalé. Ceci est contrôlé par le paramètre gamma, il faut donc reconsidérer la matrice de similarité RBF avec un gamma unitaire par défaut:

"Abstract: In kernel methods, the median heuristic has been widely used as a way of setting the bandwidth of RBF kernels."

"Large sample analysis of the median heuristic." Damien Garreau - Wittawat Jitkrittum - Motonobu Kanagawa, Max Planck Institute for Intelligent Systems (2018)
<https://arxiv.org/abs/1707.07269>

Dans ce cas, la projection est déjà intégrée à l'algorithme donc il s'applique directement aux données complètes, les métriques sont les suivantes:

| | silhouette | davies_bouldin | calinski_harabasz | adjusted_rand | nmi | time | space |
|-------|--------------|----------------|-------------------|---------------|--------------|-----------------|-----------------|
| Value | 0.1259 ± 0.0 | 2.7173 ± 0.0 | 63.0966 ± 0.0 | 0.6147 ± 0.0 | 0.7199 ± 0.0 | 0.5114 ± 0.1381 | 0.1883 ± 0.5095 |

Alors que tout indiquait que des améliorations pouvaient être espérées, les résultats sont moins bons que pour le K-Means sur ACP. Bien que l'écart-type quasi-nul soit suspect, en discutant avec d'autres groupes, la tendance semble se confirmer.

Détection d'anomalies

Auto-Encodeurs (AE)

Pour ce qui concerne les auto-encodeurs, commençons par parler du Deep Auto Encoder (DeepAE) dont le modèle se trouve dans *models.py*. Il s'agit d'un modèle d'auto-encodeur qui contient 3 couches cachées (64 -> 32 -> 16). On commence avec 64 bits pour éviter de l'overfitting sur le modèle et en raison du fait qu'on n'a pas besoin d'un modèle complexe, puisque le dataset **ecg** n'est pas très complexe. Également, commencer avec une première couche de 128 bits va certes nous donner beaucoup d'informations mais si on a plusieurs couches, il sera difficile de trouver des anomalies puisque le modèle est trop performant. Pour le choix de la taille des couches, puisqu'on a affaire à un réseau de neurones, on divise la taille par deux à chaque noeud, comme avec un arbre (64 -> 32 - 32 -> 16 - 16 -16 -16). Le même principe est appliqué sur le décodeur mais qui commence de bas en haut, au lieu de haut en bas pour l'encodeur. Après le décodeur, on a une fonction *forward(self, x)* qui reconstruit la prochaine entrée à partir de l'entrée précédente, donc $y = \hat{X}$. On applique tout cela sur une variante de l'auto-encodeur, le Denoising Auto Encoder (DenoisingAE). La différence est qu'on ajoute du bruit (5%) sur la structure de base de l'AE, donnant donc des entrées bruitées. Le but est donc de reconstruire les prochaines entrées sans le bruit, rendant l'AE plus robuste.

Parlons maintenant de l'entraînement des auto-encodeurs. L'entraînement des modèles se trouvent dans le fichier *train.py*. En comparaison avec l'*Isolation Forest*, on ne peut pas entraîner directement les data sets *X_train* (entraînement), *X_val* et *y_val* (validation) et *X_test* et *y_test* (test). Il faut créer ce qu'on appelle des DataLoaders, qui sont une combinaison d'un échantillon avec un data set et ceux-ci sont créés avec les data set d'entrées (donc *X_train*, *X_test* et *X_val*). On n'a pas besoin des dataset de sorties (*y_val*, *y_test*) étant donné qu'on fait de l'auto-supervision et qu'on reconstruit la sortie comme étant la prochaine entrée. Ensuite, il vient le temps d'entraîner les modèles d'AE. Chaque entraînement calcule la reconstruction de la sortie par méthode MSE (Mean Square Error) afin d'appliquer correctement l'algorithme d'optimisation Adam (Adaptive Moment Optimizer) sur l'AE (mauvaise reconstruction -> grosses corrections -> gros ajustements avec Adam, bonne reconstruction -> petites corrections (peu d'erreurs) -> petits ajustements avec Adam). Après avoir optimisé au besoin le modèle pour

l'entraînement, on crée des epochs (donc le nombre de fois que le modèle a été vu au complet) et des batchs (sous-ensemble du dataset). Le plus d'epochs qu'il y a, le plus de batchs qu'il aura mais plus précis sera la sortie attendue. Puisqu'on ne peut pas entraîner le modèle d'un coup avec les auto-encodeurs, il est donc logique d'appliquer l'entraînement sur des batchs tout en calculant la perte d'informations (loss) et en réoptimisant le modèle selon la quantité d'erreurs. Après la reconstruction de la sortie, on calcule les erreurs de reconstruction avec la fonction `compute_reconstruction_errors()` dans `utils.py`

Finalement, parlons des métriques de performances à calculer. Ceux-ci sont donnés après chaque entraînement d'auto-encodeur et sont calculés par une fonction `metrics_anomaly()` qui se trouve dans `utils.py`. Les métriques sont calculées une première fois avec un entraînement, puis après 10 entraînements sur chaque modèle, tout en initialisant avec un générateur de seed aléatoire pour garantir une indépendance statistique des résultats. On calcule donc, pour 1 et 10 entraînements, les métriques suivantes : *accuracy*, *recall*, *precision*, *f1_score*, *roc_auc*, *train_time* (temps d'entraînement) et *mem_usage_mb* (usage mémoire sur le processeur). On détecte également la quantiré d'anomalies selon un threshold percentage (qui est de 25% dans notre cas, correspondant à 75% des erreurs dans la reconstruction d'erreurs), ce qui influence les métriques. Voici les résultats obtenus, pour chaque métrique, en moyenne arithmétique (mean) et écart-type (std) :

Deep Auto-Encodeur (DeepAE) :

- Meilleur *accuracy* (*exactitude*) après 1 exécution (0.9730) qu'après 10 exécutions (0.9706)
- Meilleur *recall* (*rappel*) après 1 exécution (0.9914) qu'après 10 exécutions (0.9902)
- Meilleur *precision* (*précision*) après 1 exécution (0.9747) qu'après 10 exécutions (0.9730)
- Meilleur F1 score après 1 exécution (0.9830) qu'après 10 exécutions (0.9815)
- Meilleur ROC_AUC après 1 exécution (0.9512) qu'après 10 exécutions (0.9466)
- Moins de temps d'entraînement après 10 exécutions (3.2462) qu'après 1 exécution (3.6242)
- Même usage de mémoire sur le processeur pour les deux cas (0.0903 mb)

Il est bon de savoir aussi qu'après 10 exécutions, les métriques ont un écart-type, signifiant que les valeurs données sont plus stables que ceux après une exécution. Cela est utile pour comparer la stabilité entre le DeepAE et son variant DenoisingAE.

Denoising Auto-Encodeur (DenoisingAE) :

- Meilleur *accuracy* (*exactitude*) après 1 exécution (0.9814) qu'après 10 exécutions (0.9758).
- Meilleur *recall* (*rappel*) après 1 exécution (0.9970) qu'après 10 exécutions (0.9931).
- Meilleur *precision* (*précision*) après 10 exécutions (0.9798) qu'après 1 exécution (0.9765).
- Meilleur F1 score après 1 exécution (0.9883) qu'après 10 exécutions (0.9848).
- Meilleur ROC_AUC après 1 exécution (0.9552) qu'après 10 exécutions (0.9532).
- Temps d'entraînement plus long après 10 exécutions (3.4080) qu'après 1 exécution (2.8201).
- Même usage de mémoire (0.0892 mb).

Isolation Forest (IF)

Principes Algorithmiques de l'Isolation Forest

L'Isolation Forest (IF) représente une approche fondamentalement différente des auto-encodeurs pour la détection d'anomalies. Alors que les auto-encodeurs se basent sur le principe de reconstruction des données, l'Isolation Forest exploite le concept fondamental que "les anomalies sont rares et différentes". Cette différence philosophique se traduit par des mécanismes algorithmiques distincts : là où les AE tentent d'apprendre une représentation normale des données, l'IF identifie directement les points atypiques par leur facilité d'isolation dans l'espace des features.

Mécanisme d'Isolation et Architecture

L'architecture de l'Isolation Forest contraste radicalement avec la complexité des réseaux de neurones des auto-encodeurs. Contrairement aux AE qui utilisent une structure profonde avec réductions progressives (64 → 32 → 16 → 8), l'IF repose sur une forêt d'arbres de décision aléatoires. Cette simplicité architecturale présente plusieurs avantages : absence de couches

cachées à optimiser, pas de représentation latente à apprendre, et un mécanisme d'isolation basé sur des séparations aléatoires dans l'espace des caractéristiques.

Paramètres et Implémentation

Notre implémentation dans `models.py` reflète cette approche simplifiée. Le paramètre `n_estimators=100` a été choisi pour assurer une stabilité statistique sans tomber dans le sur-apprentissage, un compromis différent de la réduction dimensionnelle progressive des AE. Le paramètre `contamination=0.1` est particulièrement crucial, représentant une estimation réaliste du taux d'anomalies attendu dans les données ECG, contrairement aux AE qui utilisent un seuillage adaptatif basé sur les erreurs de reconstruction. Le `random_state=42` assure la reproductibilité, principe commun aux deux approches mais particulièrement important pour l'IF en raison de sa nature aléatoire.

Processus d'Entraînement

L'entraînement de l'Isolation Forest se distingue par sa simplicité et son efficacité. Alors que les auto-encodeurs nécessitent des `DataLoaders` complexes, des batchs, et des epochs multiples avec optimisation itérative via Adam, l'IF s'entraîne en une seule passe sur les données `X_train`. Cette absence de calcul de loss itératif et d'optimisation progressive permet une construction directe et rapide des arbres de décision, éliminant la complexité computationnelle associée au deep learning.

Avantages Computationnels

Les avantages computationnels de l'Isolation Forest sont particulièrement marqués lorsqu'on le compare aux auto-encodeurs. L'absence de préprocessing complexe contraste avec la normalisation stricte requise par les AE. Le temps d'entraînement est remarquablement court ($\sim 0.18s$ contre $\sim 3.07s$ pour les AE), tandis que l'utilisation mémoire reste négligeable (~ 0.1 MB contre ~ 483 MB). Cette efficacité s'étend également au développement, avec l'absence de réglage fastidieux d'hyperparamètres complexes comme le learning rate ou l'architecture des couches.

Résultats Expérimentaux

L'analyse comparative sur 10 exécutions indépendantes révèle des profils de performance distincts. En termes de métriques de performance, l'IF atteint une accuracy de 0.901 ± 0.033 contre 0.973 ± 0.003 pour les AE, une précision comparable (0.969 ± 0.003 vs 0.970 ± 0.004), mais montre un recall significativement inférieur (0.903 ± 0.043 vs 0.996 ± 0.005). Le F1-Score de 0.934 ± 0.024 et le ROC-AUC de 0.897 ± 0.021 confirment cette tendance.

Sur le plan computationnel, les différences sont encore plus prononcées : l'IF s'entraîne en 0.18 ± 0.01 s contre 3.07 ± 0.19 s pour les AE, avec une utilisation mémoire quasi-nulle (-0.1 ± 1.0 MB) comparée aux 483.4 ± 42.1 MB des AE.

Analyse de Stabilité et Compromis

L'analyse de stabilité révèle une variabilité plus importante pour l'IF (écart-type F1 de 0.024 contre 0.002 pour les AE), s'expliquant par la nature aléatoire de la construction des arbres. Cette variabilité contraste avec la reproductibilité supérieure des AE, bénéficiant d'une architecture fixe et déterministe.

Le compromis performance/complexité apparaît clairement : si les AE dominent sur les métriques de performance absolue, l'IF excelle en efficacité computationnelle. Le rapport performance/temps est particulièrement révélateur, avec l'IF 17 fois plus rapide pour une réduction de seulement 4.9% du F1-Score.

Avantages Distinctifs

La simplicité d'implémentation de l'IF constitue un avantage majeur. L'absence d'architecture neuronale complexe à concevoir, de réglage d'hyperparamètres de deep learning, et la transparence du pipeline d'entraînement facilitent son adoption et son déploiement.

La robustesse aux données non normalisées représente un autre atout significatif. Contrairement aux AE qui nécessitent une normalisation stricte pour assurer la convergence, l'IF fonctionne efficacement sans sensibilité particulière à l'échelle des features.

L'interprétabilité améliorée du mécanisme de décision, basé sur le concept intuitif d'isolation, offre une explication plus accessible que les représentations latentes complexes des réseaux neuronaux profonds.

Limitations et Considérations

Les limitations de l'IF apparaissent principalement dans la capture des relations complexes entre features. Son efficacité réduite pour les patterns non linéaires subtils contraste avec la capacité des AE à modéliser des relations complexes grâce à leurs fonctions d'activation non linéaires.

Le recall inférieur (90.3% contre 99.6% pour les AE) peut avoir un impact significatif dans les applications médicales critiques où la détection de toutes les anomalies est primordiale. La sensibilité au paramètre de contamination introduit également une dépendance aux hyperparamètres plus marquée que celle observée avec les AE.

Recommandations d'Application

L'Isolation Forest trouve sa pertinence maximale dans plusieurs contextes spécifiques. Pour les applications temps réel, comme le monitoring cardiaque continu nécessitant une latence minimale ou la détection en streaming avec contraintes temporelles strictes, l'IF offre des performances adaptées.

Dans les environnements contraints, que ce soit des systèmes embarqués avec ressources limitées ou des applications mobiles avec restrictions mémoire, l'efficacité computationnelle de l'IF devient déterminante.

Pour le prototypage et la validation initiale, l'IF sert de solution de baseline rapide à implémenter, permettant une évaluation rapide de la détectabilité des anomalies avant d'envisager des approches plus complexes.

Architecture Hybride et Perspectives

Une approche hybride, inspirée de la complémentarité observée entre DeepAE et DenoisingAE, pourrait optimiser le compromis performance/temps. Un premier filtrage rapide par Isolation Forest suivi d'une vérification précise par auto-encodeur permettrait de bénéficier des avantages des deux approches tout en atténuant leurs limitations respectives.

Conclusion

L'Isolation Forest incarne l'approche "efficace et simple" face aux auto-encodeurs plus "performants mais complexes". Ce choix méthodologique fondamental dépend du contexte applicatif spécifique : la performance maximale oriente vers les auto-encodeurs (F1: 98.3%), tandis que l'efficacité computationnelle privilégie l'Isolation Forest (temps: 0.18s).

Cette complémentarité méthodologique enrichit le paysage des techniques de détection d'anomalies, démontrant que chaque approche trouve sa légitimité selon les contraintes spécifiques du problème à résoudre. Dans le domaine médical des signaux ECG, où la performance et la rapidité peuvent toutes deux revêtir un caractère critique selon le scénario d'utilisation, cette diversité d'approches constitue un atout précieux pour les praticiens et chercheurs.